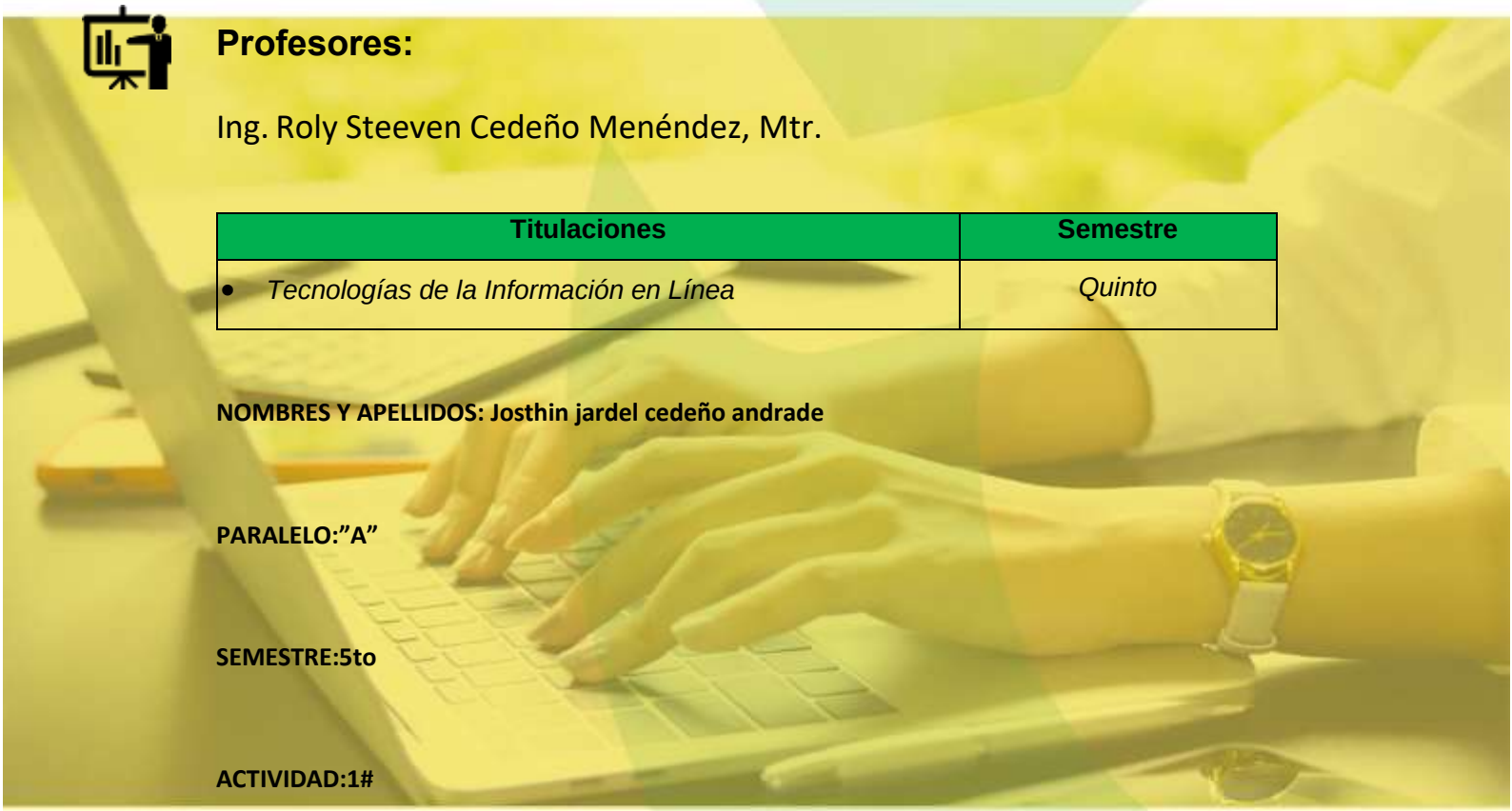
UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Josthin jardel cedeño andrade

PARALELO:"A"

SEMESTRE:5to

ACTIVIDAD:1#

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad ____

Tabla de Contenido

Estudio de caso - Sistema de control de rescate y adopción animal. 2
2. Análisis del problema: Identificar los actores principales 2
2.1 Actores principales: 2
3. Requerimientos del sistema: 3
3.1 Requerimientos no funcionales: 3
3.2 Requerimientos funcionales: 4
4. Diseño del sistema: 5
5. Aplicación de patrones de diseño 6
5.1 Patrón Singleton 6
6. Aplicación de patrones de diseño 7
6.1. Patrón Observer 7
6.3. Patrón Factory Method 8
6.4. Relación de los patrones con el sistema 9
7. Diagrama de secuencia 9
8. Secuencia del registro de rescate 10
9. Gestión de versiones mediante GitHub y Gitflow 10
9.1. Uso de GitHub 10
9.2. Modelo Gitflow 11
10. Evidencias del repositorio 13
Enlace: 14
11. Diagramas de clases y secuencia UML 14
11.1. Diagrama de clases UML 14
11.2 Diagrama de secuencia UML 15
12. Beneficios esperados del sistema 16

Actividades a desarrollar en la Unidad ____

Actividad # 1

1. Estudio de caso - Sistema de control de rescate y adopción animal.

El objetivo de este trabajo es concebir la arquitectura lógica de un Sistema de Control de Adopción y Rescate Animal, capaz de permitir el registro de animales rescatados, su clasificación por especie, la asignación a veterinarios o voluntarios a cargo y la administración de los variados estados de atención, desde el rescate hasta una eventual adopción.

En la concepción del sistema, se sugiere implementar tres modelos de diseño de software: factory fethod, observer y singleton. Estos patrones posibilitan la organización de las responsabilidades, el mejoramiento de la creación de objetos en función de lo que el sistema necesita y una comunicación más fluida entre los componentes

2. Análisis del problema: Identificar los actores principales

2.1 Actores principales:

Ciudadano o reportante: Es la persona encargada de reportar si encuentra animales abandonados, heridos o en situaciones de peligro. Es capaz de brindar detalles sobre la especie, ubicación y las condiciones del animal.

Voluntario: Es el responsable de colaborar en las actividades de rescate y seguimiento de los animales. Puede consultar los casos asignados y recibir notificaciones sobre nuevos rescates.

Veterinario: Es responsable de examinar la salud de los animales que han sido rescatados, documentar sus tratamientos y mantener al día la información sobre su recuperación.

Administrador del refugio: Es el responsable de supervisar cómo funciona el sistema, manejar las situaciones de rescate, asignar responsabilidades y crear informes según la especie de los animales.

3. Requerimientos del sistema:

3.1 Requerimientos no funcionales:

Escalabilidad: La arquitectura debe posibilitar la adición de nuevas funcionalidades y clases de animales sin que sea necesario cambiar significativamente los elementos que ya están en uso.

Mantenibilidad: El sistema tiene que tener una organización estructurada que haga más fáciles las actualizaciones, correcciones y cambios.

. Seguridad: Según el rol de cada usuario, el acceso a ciertas funciones estará restringido.

. Trazabilidad: Para documentar las modificaciones que se han hecho en sus diversas versiones, el proyecto empleará Git y GitHub..

. Reutilización: Se utilizarán elementos y patrones de diseño que sean aplicables en distintos segmentos del sistema.

. Disponibilidad: El objetivo es que el personal autorizado pueda acceder al sistema cuando lo necesite.

3.2 Requerimientos funcionales:

Registro de rescates: El sistema va a facilitar el registro de nuevos casos de animales salvados, entre los cuales se encuentran la fecha, el lugar, la especie y una descripción de su condición.

Clasificación de animales: El sistema clasificará a los animales en función de su especie, teniendo en cuenta perros, gatos y fauna silvestre etc.

Asignación de responsables: El administrador tendrá la posibilidad de encargar los casos documentados a veterinarios o voluntarios.

Actualización de estados: El personal autorizado tiene la capacidad de modificar el estado de un caso, por ejemplo: Rescatado, en terapia, en convalecencia, adoptado y cerrado.

Notificaciones automáticas: Cuando se registre un nuevo caso que necesite la atención de los voluntarios o veterinarios, el sistema les avisará.

Consulta de casos: Los usuarios que tengan autorización tendrán la posibilidad de consultar el estado y los datos de los casos registrados.

Gestión de adopciones: El personal autorizado tiene la posibilidad de documentar información vinculada al proceso de adopción de los animales que sean aptos.

Generación de reportes: El administrador tendrá acceso a informes sobre rescates, adopciones y tratamientos.

4. Diseño del sistema:

Las interacciones entre los actores y las funciones más importantes del sistema son representadas a través del diagrama de casos de uso.

Cuatro participantes son considerados en este diseño: Ciudadano, voluntario, veterinario y director del refugio.

Las interacciones más importantes son:

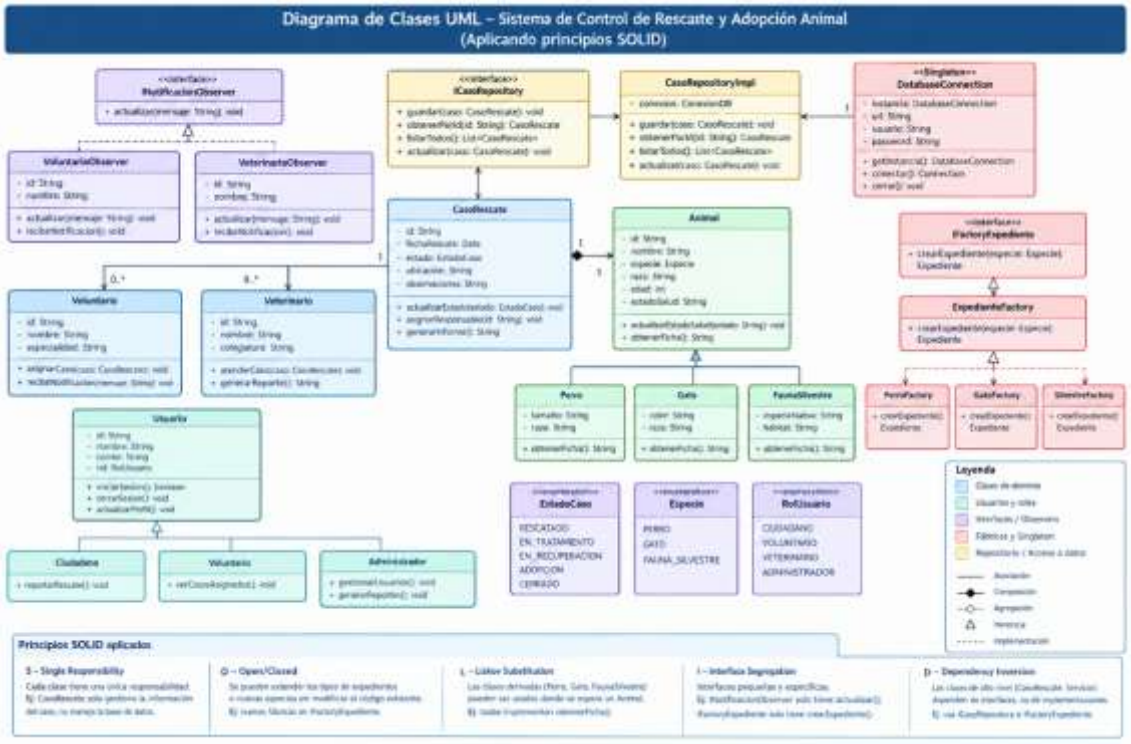
Ciudadano: Hacer informes de rescates y verificar el estado de sus reportes.

Voluntario: revisar los casos a su cargo, recibir alertas y anotar las actividades de seguimiento.

Veterinario: consultar situaciones, documentar tratamientos y actualizar la condición de salud.

Administrador: gestionar usuarios, administrar casos, designar responsables y elaborar informes

Figura 1. Diagrama de casos de uso del Sistema de Control de Rescate y Adopción Animal



5. Aplicación de patrones de diseño

5.1 Patrón Singleton

El patrón Singleton asegura que una clase tenga solo una instancia dentro de un contexto específico y brinda un acceso controlado a la misma.

Aplicación en el sistema

Este patrón se sugiere como método para el componente DatabaseConnection, que tiene la responsabilidad de concentrar las configuraciones y el acceso al mecanismo de conexión con la base de datos.

En una implementación real, este componente podría gestionar un proveedor o un grupo de conexiones en vez de conservar una única conexión física compartida.

Problema que resuelve

Elimina la generación innecesaria de varios objetos que gestionan la configuración de acceso a los datos y simplifica el control de los recursos empleados.

Justificación

Su uso posibilita la centralización de ciertas operaciones de infraestructura y el mantenimiento de una gestión más ordenada de los recursos del sistema.

6. Aplicación de patrones de diseño

6.1. Patrón Observer

Propósito

El patrón Observer posibilita que se establezca una conexión entre un objeto que produce eventos y otros objetos que desean recibir avisos cuando sucede una modificación.

Aplicación en el sistema

Se sugiere emplear este patrón para notificar de manera automática a los veterinarios y voluntarios cuando se produzca un nuevo caso de rescate.

El componente que se ocupa de la generación de notificaciones informa del evento a los observadores registrados cuando el sistema recibe un reporte.

Problema que resuelve

Con implementaciones específicas, evita que el componente de manejo de casos tenga que comunicarse directamente con cada veterinario o voluntario**.**

Justificación

El patrón facilita la inclusión de nuevos sistemas de notificación sin cambiar significativamente la lógica principal del registro de rescates.

6.3. Patrón Factory Method

Propósito

El patrón Factory Method posibilita que las clases especializadas se encarguen de crear objetos, lo que evita que otros elementos dependan directamente de las clases concretas que deben ser instanciadas.

Aplicación en el sistema

Se sugiere emplear este modelo para elaborar expedientes según el tipo de animal que ha sido rescatado.

Se consideran tres tipos de expedientes:

- ExpedientePerro.
- ExpedienteGato.
- ExpedienteFaunaSilvestre.

Cada fábrica concreta se encargará de crear el expediente correspondiente, respetando una interfaz común.

Problema que resuelve

Evita concentrar la creación de todos los tipos de expedientes en un único componente con numerosas condiciones.

Justificación

Permite que se añadan nuevas categorías de animales y promueve una estructura que es flexible, ordenada y reutilizable.

6.4. Relación de los patrones con el sistema

Los tres patrones desempeñan roles distintos en la arquitectura sugerida**.**

Factory Method sistematiza la creación de expedientes conforme a las características de los animales; Observer posibilita la comunicación automática de eventos nuevos; y Singleton hace posible centralizar el manejo de un recurso compartido.

Su combinación ayuda a dividir responsabilidades y permite que el sistema pueda ser modificado en el futuro.

7. Diagrama de secuencia

El diagrama de secuencia muestra la secuencia en la que interactúan los distintos componentes a lo largo de una operación del sistema.

El objetivo de esta actividad es ilustrar el procedimiento para registrar un nuevo rescate.

8. Secuencia del registro de rescate

Paso 1. El ciudadano ingresa la información del animal rescatado mediante el formulario correspondiente.

Paso 2. El controlador recibe los datos y solicita la creación del caso al servicio de gestión.

Paso 3. El servicio valida la información proporcionada.

Paso 4. Se utiliza la fábrica correspondiente para crear el expediente según la especie del animal.

Paso 5. El servicio registra la información mediante el repositorio de casos.

Paso 6. Se genera un evento de nuevo rescate y se notifica a los observadores registrados.

Paso 7. Los voluntarios o veterinarios correspondientes reciben la notificación.

Paso 8. El sistema confirma que el caso fue registrado correctamente.

9. Gestión de versiones mediante GitHub y Gitflow

9.1. Uso de GitHub

GitHub es una plataforma que permite alojar repositorios de proyectos y colaborar en el control de versiones mediante Git.

Para esta actividad se utilizará un repositorio público destinado a almacenar los archivos del diseño, la documentación y los posibles componentes de implementación del sistema.

Nombre propuesto del repositorio: sistema-rescate-adopcion-animal

Enlace del repositorio: [Agregar enlace real de GitHub].

9.2. Modelo Gitflow

Gitflow es un modelo de organización laboral que emplea ramas distintas para dividir el proceso de desarrollo de nuevas funciones y las versiones estables.

La estructura siguiente se plantea para este proyecto.:

Rama master: Contendrá las versiones estables del proyecto.

Rama develop: Se utilizará para integrar los avances realizados durante el desarrollo.

Ramas feature/: Permitirán trabajar en funcionalidades o componentes específicos de forma independiente.

Como ejemplos de ramas de desarrollo se proponen:

- feature/registro-rescates
- feature/notificaciones
- feature/expedientes
- feature/diagramas-uml

9.3. Flujo de trabajo propuesto.

Al principio, se formará el repositorio y se definirán las ramas develop y master.

Después, se establecerán ramas feature/ para llevar a cabo o documentar cada elemento. Una vez que una funcionalidad está terminada, sus modificaciones se incorporan en develop a través de una solicitud de incorporación de cambios (pull request).

Una vez que se hayan evaluado los progresos y que una versión estable esté lista, se añadirán los cambios necesarios a master.

Este método posibilitará que se mantenga el orden de las versiones y facilitará el seguimiento de los cambios efectuados.

9.4. Justificación del uso de Gitflow

Al usar Gitflow, se logrará distinguir las versiones estables del sistema de los cambios que se están desarrollando.

Asimismo, permitirá que se reconozcan las funciones integradas, se revisen los cambios y se recuperen versiones previas cuando sea requerido.

Este modelo posibilita la implementación de prácticas empleadas en proyectos profesionales de desarrollo del software, a pesar de que la actividad se lleva a cabo individualmente..

10. Evidencias del repositorio

Se presentarán capturas de pantalla auténticas del esquema de ramas y del historial de commits del repositorio para evidenciar la implementación del modelo Gitflow.

Figura . Repositorio público del proyecto en GitHub.

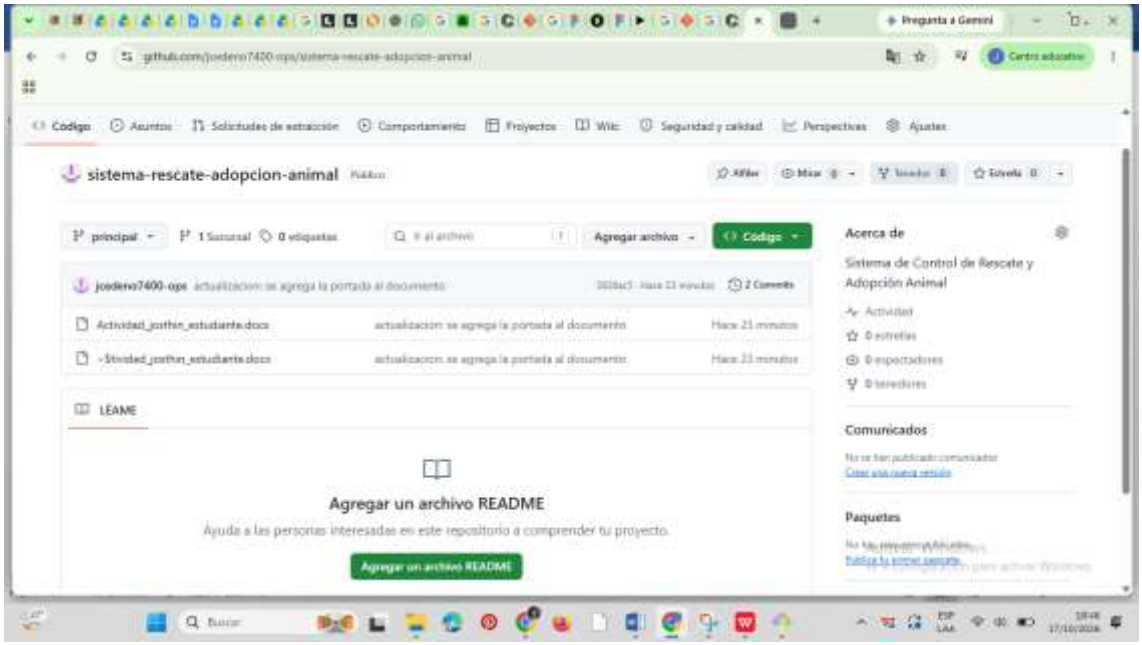
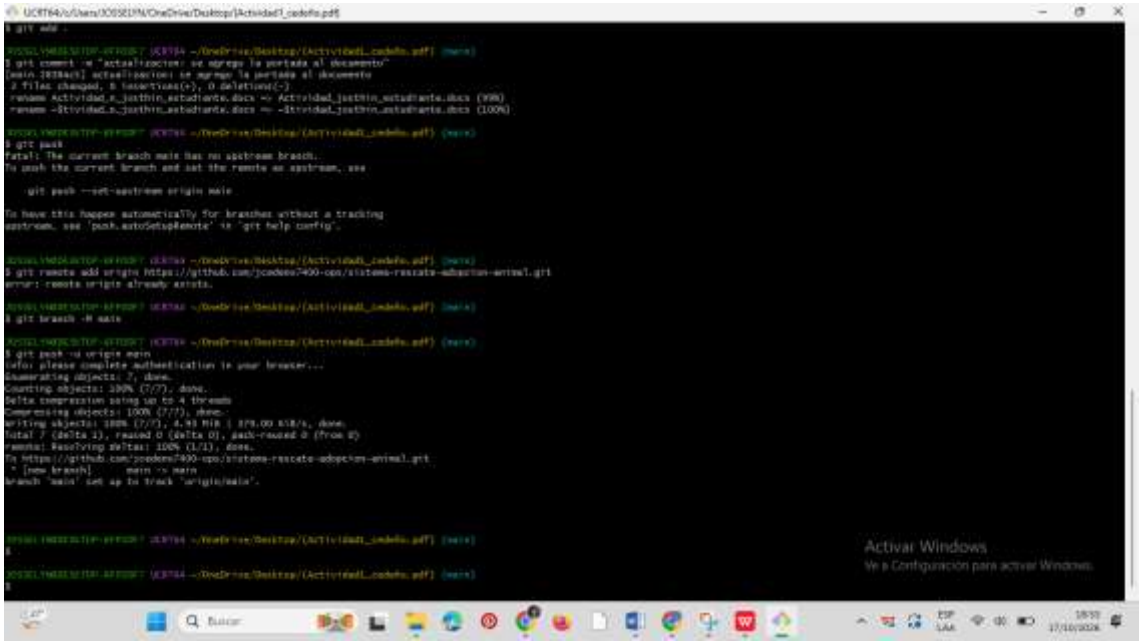


Figura . Historial de commits realizados.



Enlace:

https://github.com/jcedeno7400-ops/sistema-rescate-adopcion-animal

11. Diagramas de clases y secuencia UML

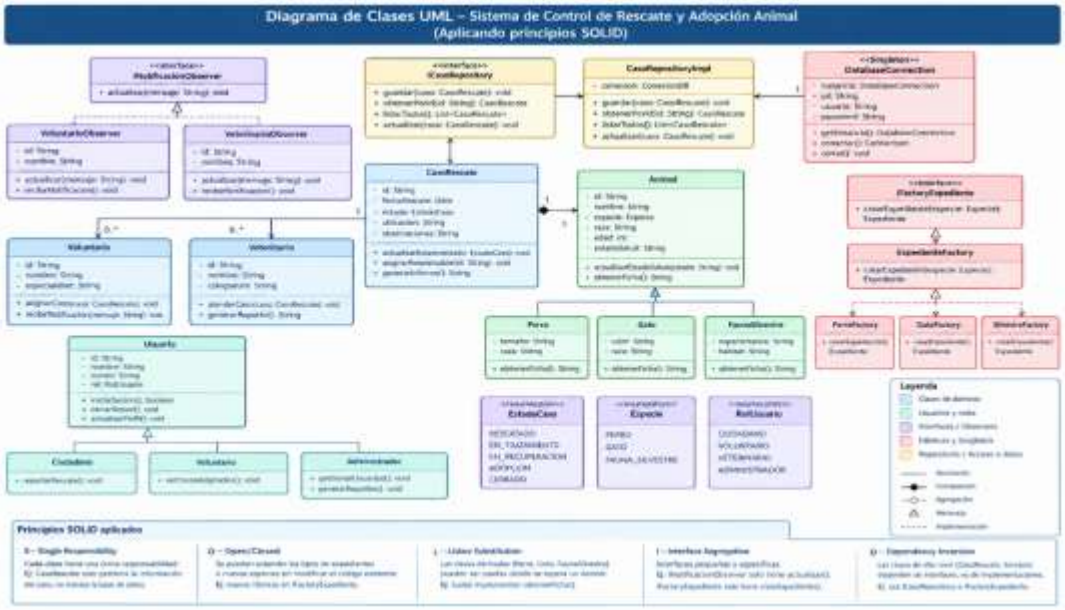
Los diagramas UML permiten representar la estructura y el funcionamiento del Sistema de Control de Rescate y Adopción Animal. En este diseño se utilizan diagramas de clases y de secuencia para mostrar cómo se aplicarían los patrones Singleton, Observer y Factory Method.

11.1. Diagrama de clases UML

El diagrama de clases representa los componentes principales del sistema, incluyendo sus atributos, métodos y relaciones. Además, permite identificar las clases e

interfaces que participan en la aplicación de los patrones de diseño y los principios SOLID.

Figura, Diagrama de clases UML del Sistema de Control de Rescate y Adopción Animal.

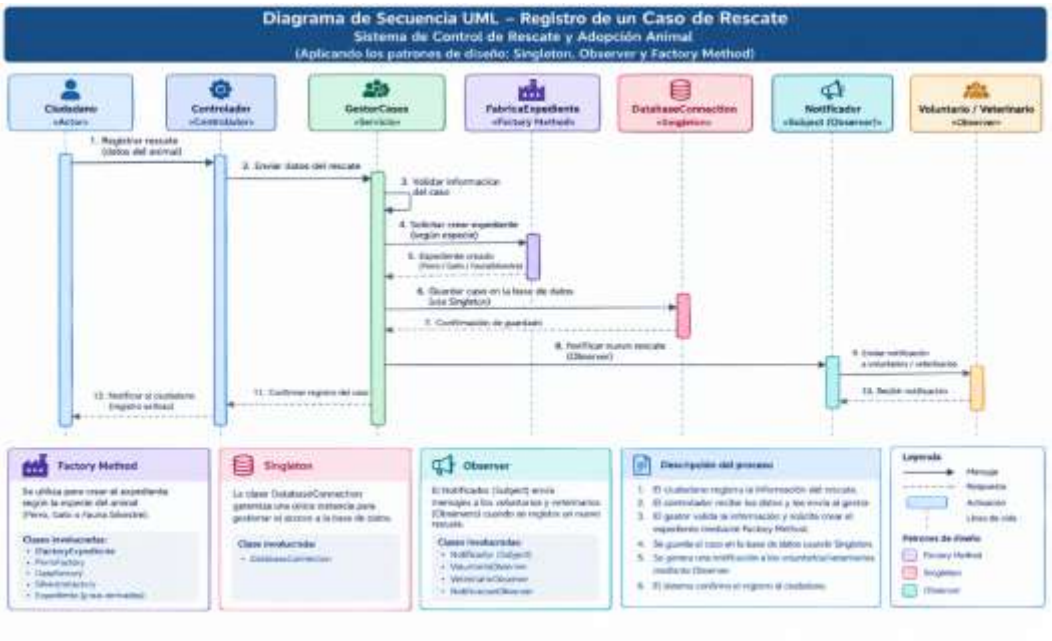


11.2 Diagrama de secuencia UML

El diagrama de secuencia ilustra cómo interactúan los componentes durante el registro de un rescate animal nuevo.

Factory Method participa en la generación del expediente con base en la especie del animal, Singleton centraliza el acceso al módulo de conexión con los datos y Observer facilita la comunicación a los veterinarios o voluntarios sobre un nuevo caso que ha sido registrado.

Figura . Diagrama de secuencia del registro de un caso de rescate.



12. Beneficios esperados del sistema

A través de una arquitectura estructurada, la propuesta de diseño tiene como objetivo optimizar la organización y el seguimiento de los casos de rescate de animales.

La centralización de la información, la categorización de los animales por su especie, la distribución ordenada de las responsabilidades y el monitoreo del progreso en sus diferentes etapas son algunos de los principales beneficios esperados.

Además, el uso de principios SOLID y patrones de diseño posibilitaría disminuir el acoplamiento entre los distintos componentes y simplificar la adición de nuevas características.

Usar Gitflow ayudaría a que el registro de las modificaciones sea organizado y a que la trazabilidad de las versiones del proyecto mejore.